

How a Senior Engineer Scaffolds an App Foundation

The feature-scaffolding manual assumes something already exists to build a feature on top of: a repo, a data layer, an auth story, a deploy pipeline, conventions. This is the manual for the step before that — building the thing every future feature will inherit.

Get this layer wrong and every feature scaffolded on top of it inherits the mistake, multiplied by however many features you ship. Get it right and Phase 3 of the feature manual ("scaffold the vertical slice") becomes routine, because the slots already exist — you're just filling them.

Two kinds of scaffolding happen here too:

Scaffolding as project generation — `create-next-app`, `nest new`, `rails new`, `django-admin startproject`, a Flutter project template. This gives you an empty-but-wired repo.

Scaffolding as foundation-building — deciding and wiring the handful of structural decisions that are expensive to change later (data boundary, auth model, module boundaries, deploy path) and proving them end-to-end with a throwaway "hello world" feature before any real feature exists. This is the more important skill; the generator just saves typing inside it.

Phase 0 — Frame the app before touching code

You cannot scaffold a foundation for a product you haven't defined at the system level.

Write the product statement: who is this for, what's the core loop, what does the app do that justifies its own codebase (rather than being a feature of something else)?

Define the boundary of v1. Not the full vision — the smallest set of capabilities that make the product real to a first user. Everything else is a non-goal for the foundation, even if it's on the roadmap. A foundation built for six modules you haven't validated yet is over-engineering; a foundation that can't grow a second module without a rewrite is under-engineering. Aim for the latter to be a deliberate, named risk, not an accident.

Identify the unknowns that would change the foundation if you got them wrong, and spike them before committing: Which auth provider fits? Can the target hosting run this stack? Does the core data shape actually support the query patterns the product needs? Throw the spike code

away, keep the answer. An unspiked unknown baked into the foundation is expensive precisely because everything else gets built on top of it.

Decide the constraints that shape everything downstream: expected scale (10 users or 10,000?), team size (solo, or others joining?), deployment target, budget for infra, and how long this foundation needs to last before a rewrite is acceptable. These answers determine how much foundation-building is worth doing now versus deferring.

Phase 1 — Foundational design

This is Phase 1 of the feature manual, but answered once, at the level that every feature will sit inside.

Model the core domain, not one feature's data. What are the 4-8 entities the whole product revolves around, how do they relate, and where are the boundaries between them? You're not designing every table — you're deciding the shape new tables will need to respect. Decide the default conventions here too: ID strategy (UUID vs auto-increment), timestamp columns, soft delete vs hard delete, naming conventions — so every future migration doesn't relitigate them.

Design the module/layer boundaries. Where does business logic live, separate from HTTP/DB concerns? What's the folder structure a new feature drops into? A clear answer here is what makes Phase 3 of feature-building ("data → repository → service → endpoint → frontend") a repeatable motion instead of a fresh decision each time.

Decide the API contract style the whole app will use — REST vs GraphQL vs RPC, response envelope shape, error format, pagination convention, versioning approach. Every endpoint any feature ever adds should look like it was written by the same person.

Decide auth and authorization once. Who can be a user, how do they authenticate (sessions, JWT, OAuth, magic link), how are roles/permissions modeled, and how does every future protected route check them the same way? Auth is Do-Not-Vibe territory — this gets deliberate human design, not generated boilerplate accepted on faith.

Sketch the app shell. Navigation structure, layout, global state (auth/session, theme, feature flags), design system starting point (component library, styling approach) — the frame every screen will sit inside, not any one screen's content.

Flag the Do-Not-Vibe surface for this app specifically. Payment math, webhooks, migrations, auth/reset flows, production secrets are the universal list — but name the app-specific additions too (e.g. anything touching money for a finance product, anything touching client data for a booking product).

Sequence the foundation work by dependency. Repo and tooling first, then data/auth (everything depends on them), then the app shell, then the first throwaway vertical slice that proves it all connects.

The output of this phase is the architecture map every future feature's Phase 1 will reference instead of re-deriving.

Phase 2 — Repo, environment, and tooling setup

The one-time housekeeping that every future feature branch inherits for free.

Initialize the repo with the generator appropriate to the stack, then strip what you don't need rather than accumulating unused boilerplate.

Set up environments — local, staging, production — and the config/secrets strategy for each (.env conventions, secrets manager) before any feature needs a secret.

Set up the database and connection/migration tooling (ORM or query builder, migration runner), and get an empty schema applying cleanly in every environment.

Set up CI — a pipeline that runs lint, type-check, and tests on every push, even when there's nothing to test yet. Wiring this before feature work starts means every feature branch is covered from its first commit, not retrofitted later.

Set up lint, format, and type-check configs and commit them — this is the shared contract every feature's Phase 6 ("quality gates") checks against.

Set up the branching and commit convention (trunk-based, main/develop, conventional commits or not) so nobody has to decide this per-feature.

Set up the feature-flag system if the product will need staged rollouts — wiring the mechanism now means Phase 2 of the feature manual ("set up a feature flag") is a one-line addition, not new infrastructure each time.

Set up local seed data / fixtures tooling — a script that gets a fresh clone to a working local state in one command.

Phase 3 — Scaffold the vertical slice of the app itself

Same principle as feature-scaffolding, one level up: build a thin slice through every layer of the app, not of any one feature, and get it deployed before building real functionality.

Data layer. Apply the core schema decided in Phase 1 — auth tables plus one placeholder domain table. Run the migration in every environment.

Auth end-to-end. Sign-up, login, session/token issuance, and one protected route that rejects unauthenticated requests. This is the piece every future feature will assume already works — prove it now, deliberately, by hand.

API layer. The shared middleware stack — request validation, error handling, response serialization, logging — wired once, applied everywhere, plus one real endpoint that uses it.

Frontend shell. Routing, the auth-aware layout (logged-in vs logged-out states), a global API client with the error/loading conventions every feature's frontend will reuse.

One true page. A single real screen — often account/settings or a trivial dashboard — wired end-to-end through every layer above, deployed to staging.

Deploy pipeline. CI builds it, it deploys to staging automatically, and you can reach the live URL and log in. This is the proof the foundation actually works, not just compiles.

At the end of Phase 3 you have an app that does nothing product-specific but is fully wired: auth works, one page renders real data from a real database, deploys are automatic, and CI is green. Every feature from here on is addition, not infrastructure.

Phase 4 — Harden the foundation before features land

Before the first real feature branches off, close the gaps a thin slice skips.

Error handling and observability at the app level — a global error boundary on the frontend, centralized error logging on the backend, and a monitoring/alerting hookup (even a basic one) so failures in feature code surface automatically.

Authorization scaffolding — the reusable guard/middleware pattern every protected feature route will call, proven on the one real endpoint from Phase 3.

Base UI kit — the handful of shared components (button, input, layout primitives, loading state, empty state, error state) every feature's frontend will compose from, so no feature reinvents them.

Testing scaffolding — the test runner configured for unit, integration, and e2e, with one example of each committed, so a feature's Phase 5 ("testing pyramid") has a pattern to copy instead of a blank page.

API documentation scaffolding — even a minimal auto-generated spec, so the contract every feature adds to is visible and versioned from day one.

Performance baseline — indexes on the core tables you know every feature will query against (users, the primary domain table), and pagination conventions decided before the first list endpoint needs them.

Phase 5 — Foundation-level quality gates

Before this is a base other work builds on:

Lint, format, type-check all green on the empty-but-wired app — the baseline every future PR is diffed against.

CI pipeline green end-to-end, including the deploy step.

Self-review the whole repo structure as if reviewing someone else's foundation: is the module boundary from Phase 1 actually followed by what got built? Is anything already leaking across layers it shouldn't?

README and architecture docs written — the map a new feature (or a new contributor) reads before writing the first line of feature code: folder structure, conventions, how to run locally, how auth works, where the Do-Not-Vibe lines are.

Phase 6 — Release the foundation itself

Deploy to staging and production even though there's no real feature yet — prove the pipeline works under real deploy conditions, not just in CI.

Tag the release (v0.1.0 or similar) as the point every feature branches from.

Smoke-test production — sign up, log in, hit the one real page, confirm monitoring is actually receiving events.

Write the contributor/onboarding note — what "done" looks like for a feature built on this foundation, pointing back at the feature-scaffolding manual for the per-feature loop.

The whole thing as a checklist

Frame: product statement · v1 boundary · non-goals for the foundation · spike unknowns · scale/team/deploy constraints

Design: core domain model + conventions · module/layer boundaries · API contract style · auth & authorization model · app shell sketch · app-specific Do-Not-Vibe surface · task order

Setup: repo init · environments (local/staging/prod) · database + migrations · CI pipeline · lint/format/type-check configs · branching convention · feature-flag system · seed data tooling

Scaffold (vertical slice of the app): schema + auth tables → auth end-to-end → shared API middleware → frontend shell + routing → one real page → CI deploy to staging

Harden: global error handling + observability · authorization guard pattern · base UI kit · testing scaffolding + one example per layer · API doc scaffolding · performance baseline on core tables

Quality: lint/format/type-check green · CI green · structural self-review · README + architecture docs

Release: deploy staging + production · tag the release · smoke test · contributor onboarding

note

The one idea to keep

Build the app's skeleton thin and vertical before any feature exists, then let every feature thicken it. The foundation isn't a pile of infrastructure decisions made in the abstract — it's the same "one true path end-to-end" discipline from feature-scaffolding, applied to the app itself: auth, one real page, one real deploy, proven to connect. Every feature you scaffold afterward plugs into slots that already exist, instead of each one relitigating the decisions this phase should have settled once.